
Request Reply
Remote Method Invocation
Remote Service Invocation

Middleware layers

Applications

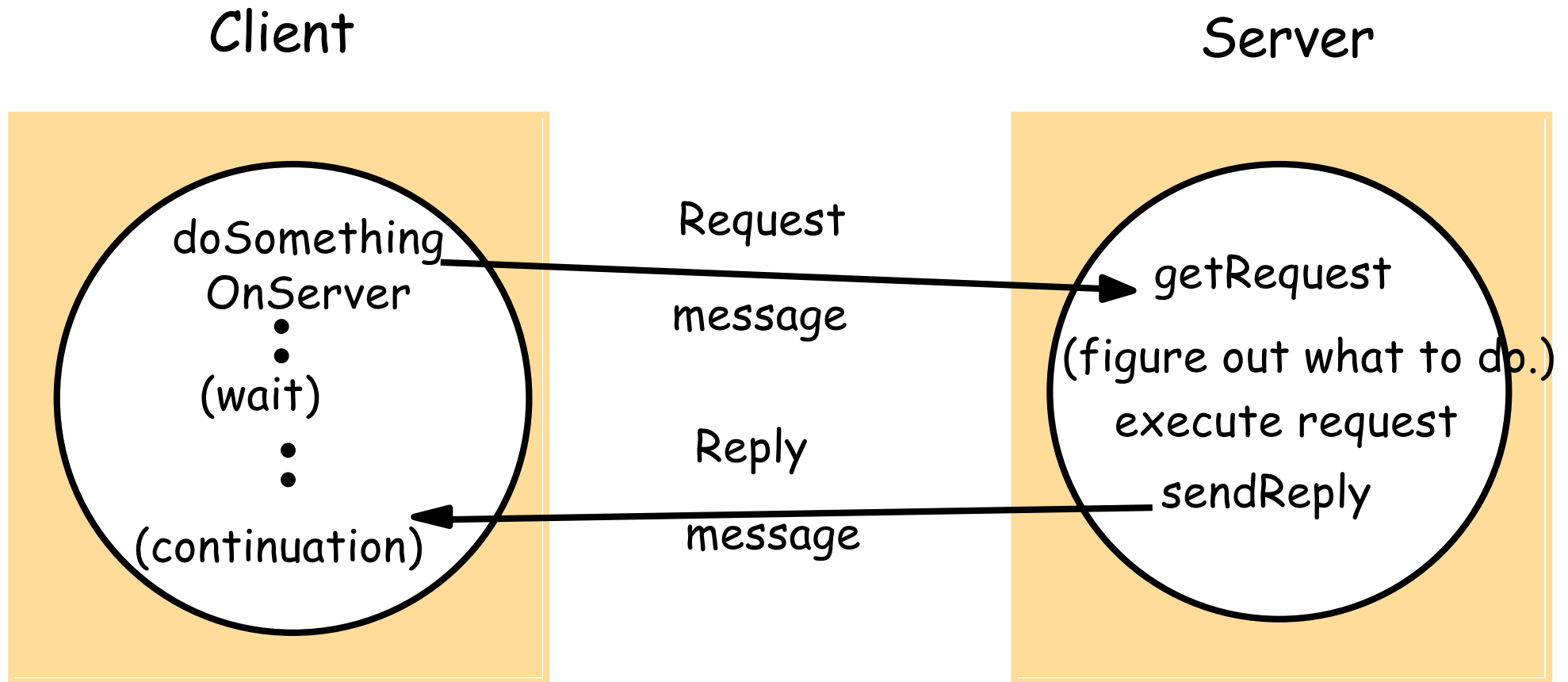
Request / Reply approaches: HTTP, RMI, RPC, WebService

External data representation

Middleware

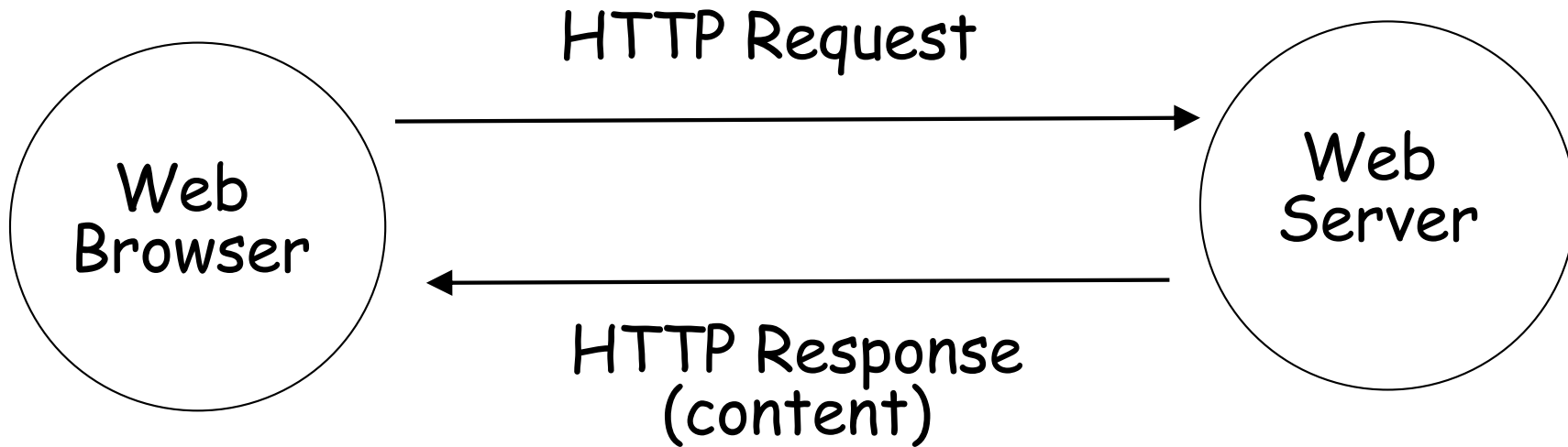
Operating System/Basic Communication (TCP/UDP)

Client-Server communication



HTTP: Hypertext Transfer Protocol

- ❑ On top of TCP



Features

- ❑ Request is on a Resource
- ❑ Address of Resource
 - ❑ Uniform Resource Identifier / Uniform Resource Locator (URL)
 - ☆ Location: Logical address name plus potentially port
 - ☆ Unique Name within location
 - Can be a file (static content)
 - Can be a program
 - ☆ <http://www.cs.mcgill.ca/~kemme/cs512/index.html>
- ❑ Simplest:
 - ☆ Get Request to retrieve static content

Advanced Requests

❑ Methods on the resource

★ Get

- On data: return content
- On program: run program and return result
- *example.com/page1/form.php?name1=value1&name2=value2*

★ Put

- URL already exists: override
- New URL: create

★ Post

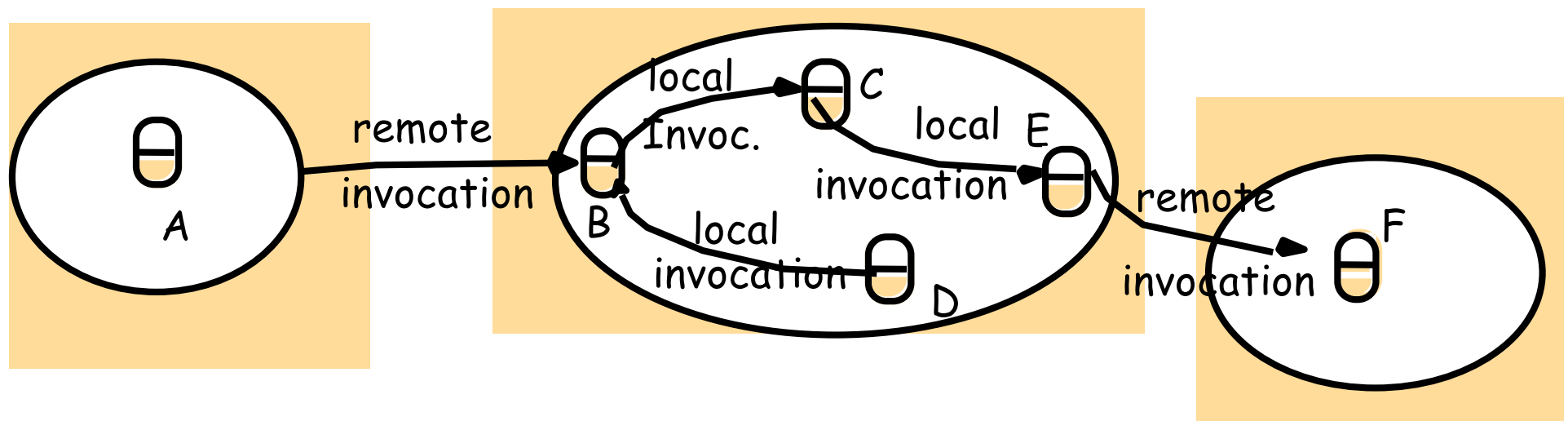
- Add data to the resource (e.g. a mailing list, database...)

★ Parameters: program parameters

- E.g., data entered in a form...
- Syntax can get complicated!

❑ Textual Format

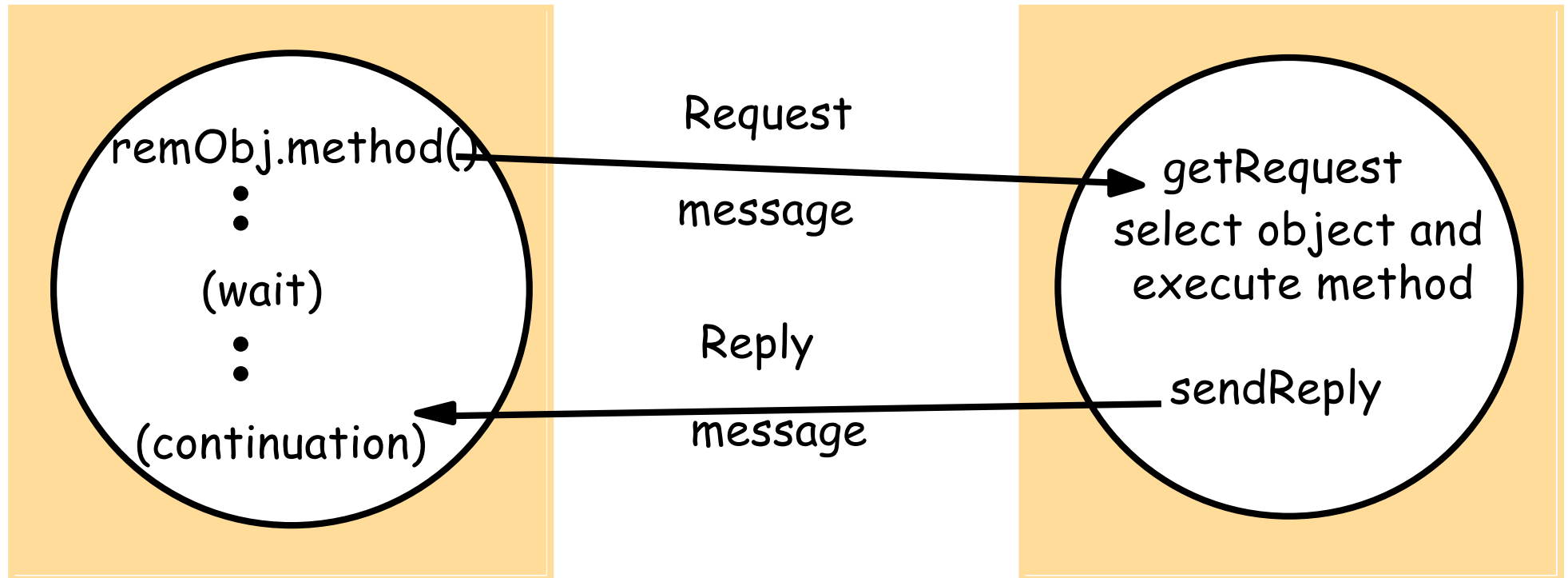
Remote Method Invocation



Client-Server communication

Client

Server



Programming Models for Distributed Applications

❑ Remote Procedure Call (RPC)

- ☆ Client calls a procedure in a server running on a different machine
- ☆ Idea: call to remote procedure should look like a call to local procedure
- ☆ Theory since 1976, first implementations in the early 80's

❑ Remote Method Invocation (RMI)

- ☆ Same as RPC in OO-world
- ☆ Object calls method of another object residing on a different machine
- ☆ First: implementations had same architecture than RPC
- ☆ Current developments: more features and variations
- ☆ Trend to truly distributed computing

❑ Remote Service Call

- ☆ Server describes service interface
- ☆ Calls via http (web service)
- ☆ Message exchange via XML/JSON...

Review of OO-programming

- ❑ An object encapsulates both *data* and *methods*
- ❑ Objects are accessed via *object references*
- ❑ Method Invocation can cause
 - ★ the state of the called object to change
 - ★ further invocations on methods in same or other objects
- ❑ Access to variables (direct/indirect)
 - ★ direct (e.g., public): not suitable in distributed world
 - ★ indirect (e.g., via getter/setter methods): encapsulation
 - ★ Assumption from now: variables of an object can only be accessed via method calls

Review of OO-programming

❑ *Exceptions*

- ★ are thrown when an error occurs.
- ★ can be caught (redirection to specific code handling the exception)
- ★ or delivered to caller.

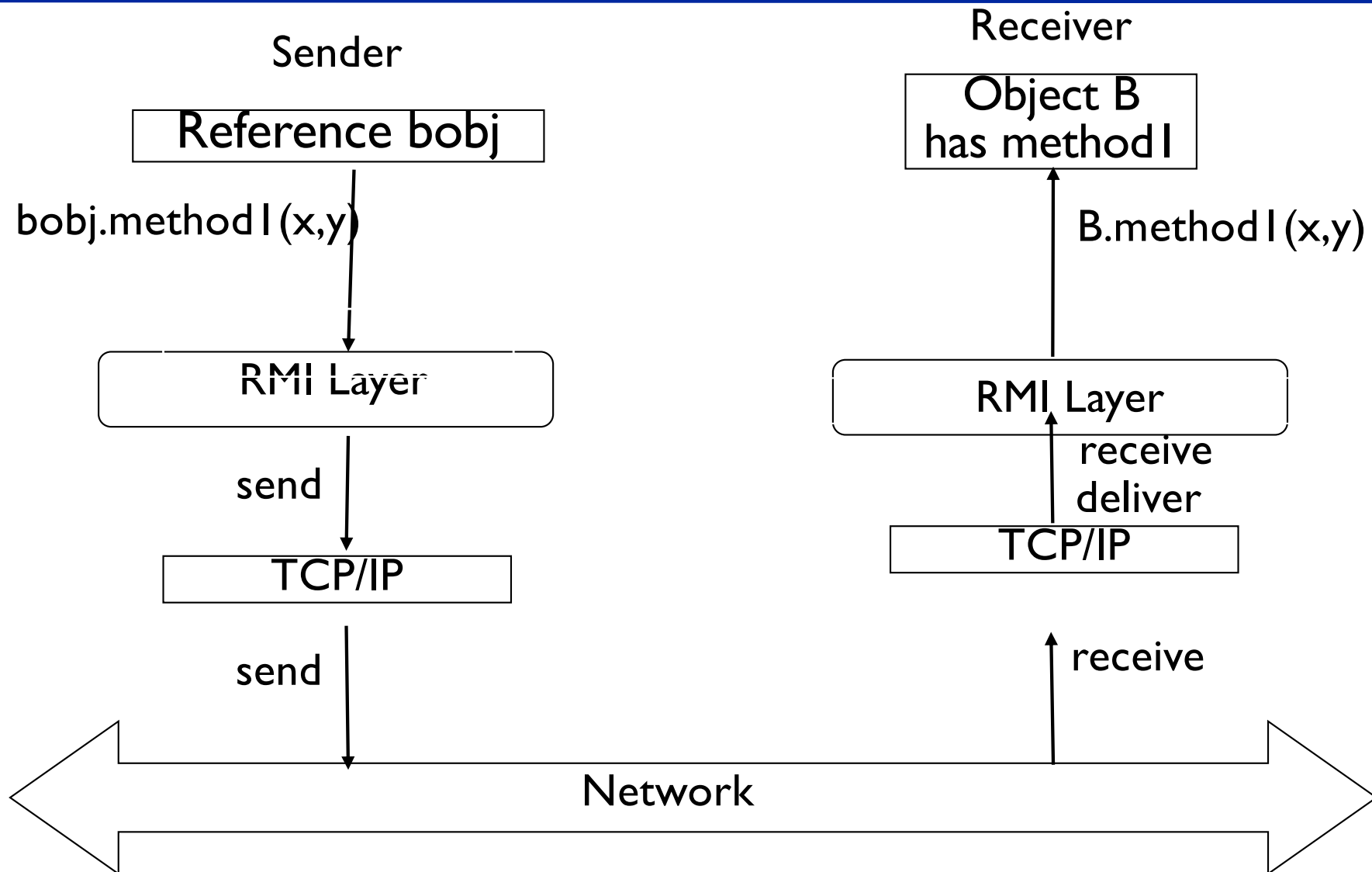
❑ *Interfaces:*

- ★ provide signatures of a set of methods
- ★ description of input and output parameters of the methods

❑ *Garbage collection*

- ★ frees space when objects are no longer needed

RMI Architecture



RMI Interface

- ❑ *Remote Interface File* of an object class
 - ☆ Interface file for the set of methods that can be called remotely
 - ☆ object class can have further methods that can only be called locally
- ❑ Method description in remote interface file
 - ☆ input and output parameters are specified
 - ☆ a parameter can be an object

Interface File in Java RMI

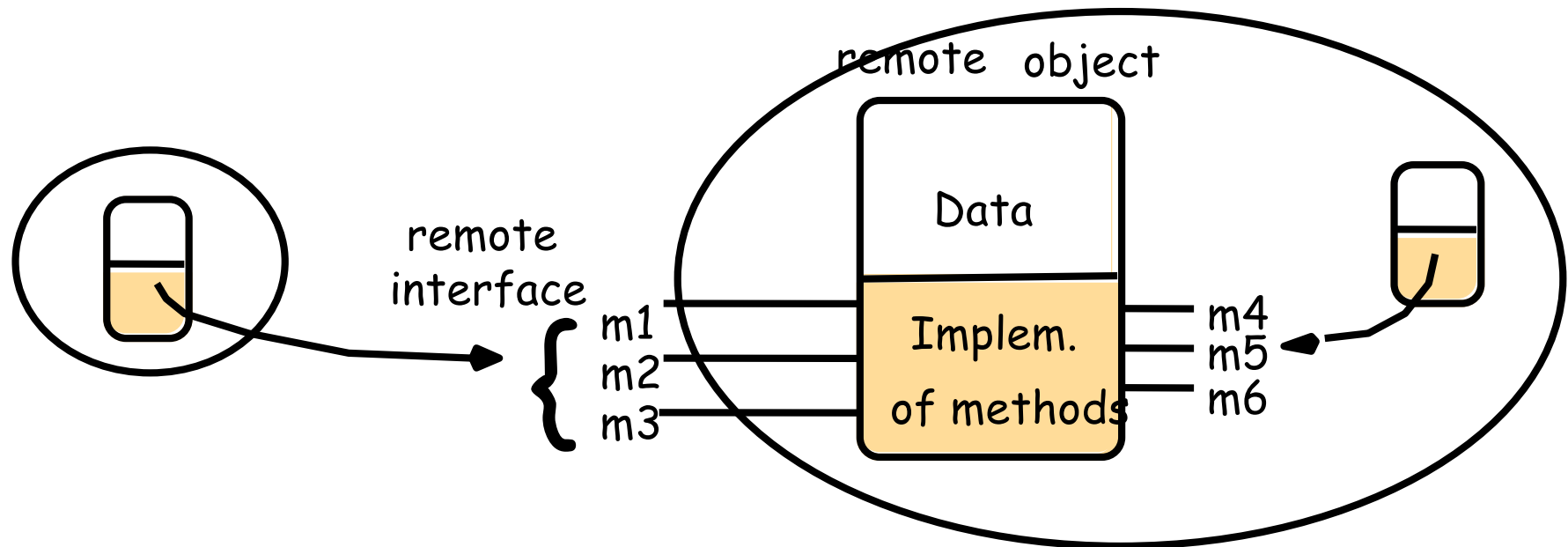
```
import java.rmi.Remote;  
import java.rmi.RemoteException;  
public interface Account extends Remote {  
    public float addtoBalance(float amount) throws RemoteException;  
    public float getBalance() throws RemoteException;    }
```

- ❑ By extending `java.rmi.Remote`, this interface's methods can be called from any virtual machine. Any object that implements this interface becomes a remote object.
- ❑ As a member of a remote interface, the `getBalance` method is a remote method. Therefore the method must be defined as being capable of throwing a `java.rmi.RemoteException`.
- ❑ This exception is thrown by the RMI system during a remote method when communication failure or a protocol error has occurred.
- ❑ Any code making a call to a remote method needs to handle this exception by either catching it or declaring it in its throws clause.

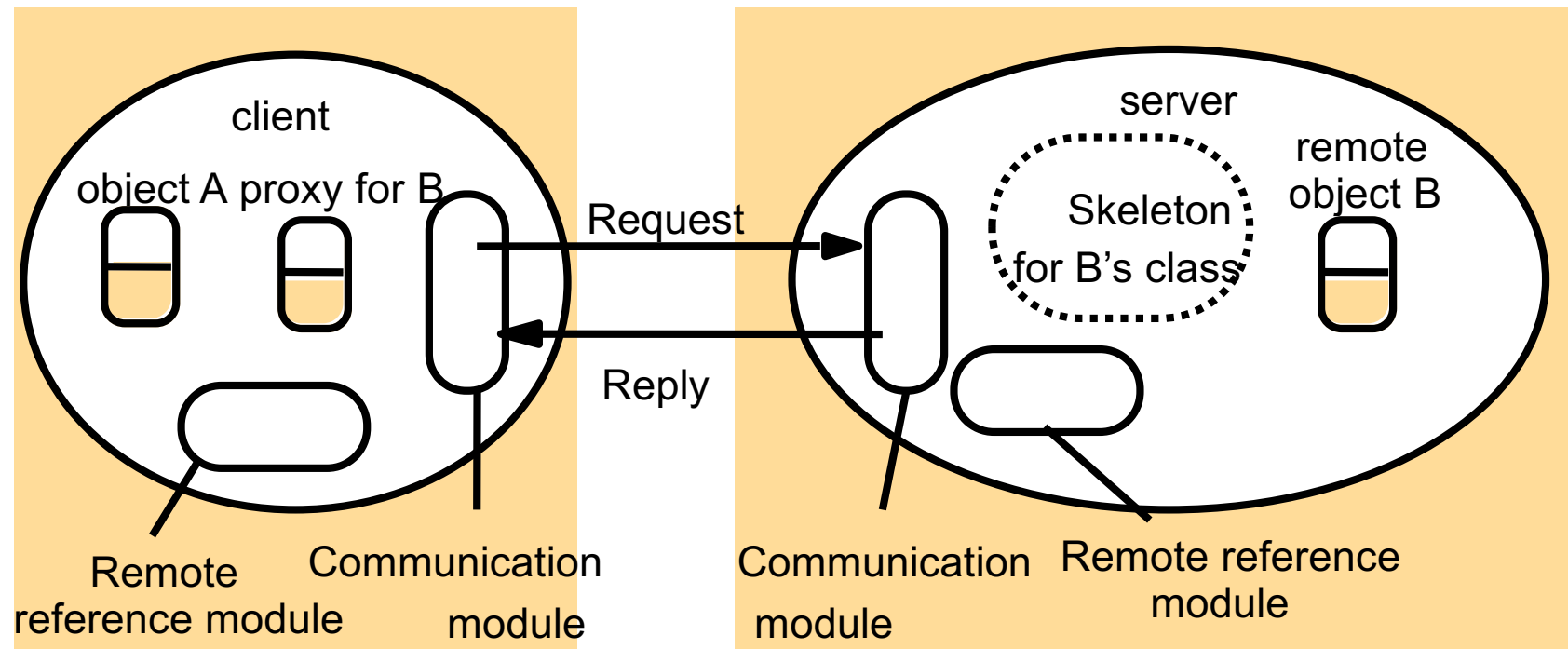
Interface Definition Languages

- ❑ Java has the concept of interfaces and hence, the remote interface can be described as usual + extending interface to be REMOTE
- ❑ In general interfaces are described with Interface Definition Languages
- ❑ Example RMI
 - ★ CORBA IDL
- ❑ Example RPC
 - ★ Google Protocol RPC Service Interface (.proto file)

A remote object and its remote interface



RMI Components



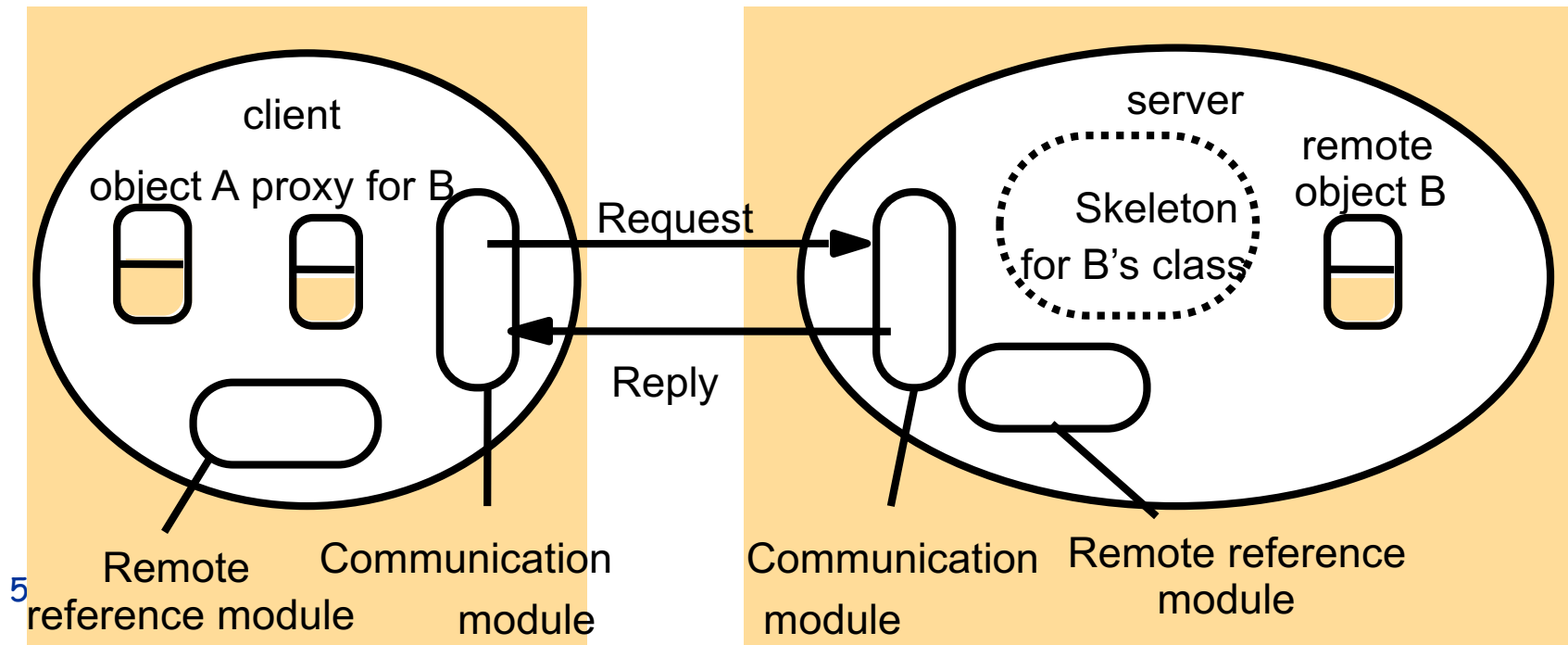
RMI Objects

- ❑ Implementation of local and remote methods

 - ☆ only at server

- ❑ Proxy object at client (stub)

 - ☆ contains proxy methods for all methods of the remote interface



RMI Execution

❑ On the client

☆ object A calls method of remote object B

- it internally calls the proxy-method of the local proxy object.
- proxy also called *stub*
- i.e., object A has an object reference to the local proxy object

RMI execution

❑ On the client

☆ Within proxy-method and underlying communication module (RR module)

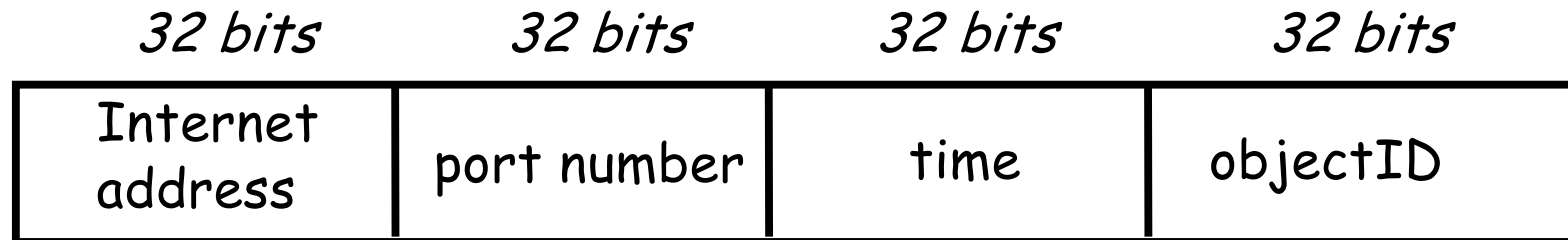
- marshals *request* message containing
 - ▲ *remote reference* to the remote target object,
 - ▲ its own `methodID`
 - ▲ and the arguments
- sends request message to server
- awaits reply message
- unmarshals reply
- returns the results to the invoker

RMI Execution

❑ On the server

- ☆ RMI communication module receives message
- ☆ unmarshals message
- ☆ determines local object B given the reference and the methodID
- ☆ invokes corresponding method on real object B
- ☆ marshals the result (together with any exception) into result message
- ☆ sends result message back to RMI layer on caller

Remote Object Reference



- ❑ In non-distributed execution, objects can be accessed via object reference.
 - ☆ In Java, a variable that appears to hold an object actually holds a reference to that object.
- ❑ An object A can invoke the method of a remote object B, if it has access to B's remote object reference.
- ❑ Generally: A remote object reference is an identifier that can be used throughout a distributed system to refer to a particular unique remote object.
- ❑ Note: In some RMI online information the remote object reference actually refers to a stub/proxy instance
- ❑ Conceptually like a URL

Remote reference module

❑ Maintains remote object table

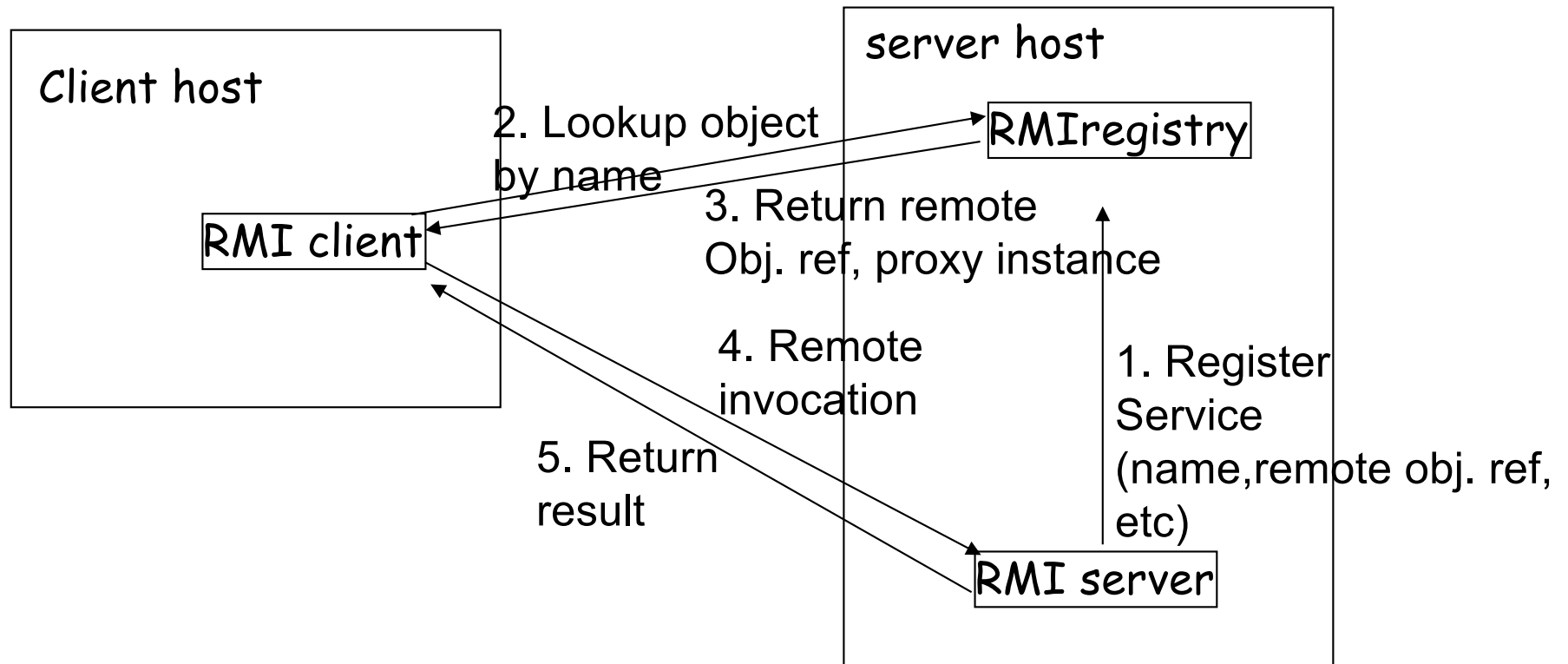
☆ entry

- remote object reference and local proxy reference (on client)
- remote object reference and real object reference (on server)

❑ Some Tasks

- ☆ On a server: when a remote object reference arrives in a request message it looks for the corresponding local object reference which points to the real object
- ☆ On a client: when a remote object reference arrives in a reply message it looks for the proxy object.
- ☆ When a remote object is to be passed as argument or result for the first time, the remote reference module creates a remote object reference and adds it to the table

A basic Java RMI call



The binder: RMI Registry

- ❑ An instance of the Rmiregistry must run on every computer on which a server wants to export a remote interface.
- ❑ It maintains a table mapping textual references to remote objects hosted on that computer.
 - ☆ `//computeName:port/objectName`
 - ☆ `computeName` is the host name on which the registry resides and `port` is the port the rmiregistry listens on

RMI Registry Interface

- ❑ Void rebind(String name, Remote obj)
 - ☆ Register object by name
 - ☆ Override previous registration
- ❑ Void bind (String name, Remote obj)
 - ☆ Register an object by name
 - ☆ If existent throw exception
- ❑ Void unbind(String name, remote obj)
- ❑ Remote lookup(String name)
 - ☆ Used by client
 - ☆ Remote object reference is returned
- ❑ String[] list()
 - ☆ Show all names bound in this registry

Method Parameters

- ❑ Input and output parameters can be
 - ☆ Primitive types (pass by value)
 - ☆ Remote objects (pass the remote object reference)
 - If the recipient of a remote object reference does not have the object proxy, then the object proxy is automatically downloaded
 - ☆ Local objects. They are passed by copy, using object serialization (the object must implement the `java.io.Serializable` Interface).
 - If the recipient of an object passed by value does not already possess the class of the object, its code is automatically downloaded.

Server Implementation

```
import java.rmi.registry.Registry;
import java.rmi.registry.LocateRegistry;
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;

public class AccountImpl implements Account {

    float balance;
    int ID;

    public AccountImpl(float b, int id){ // initialize an account};
    public float getBalance() { real impl } ;
    public float addToBlance (float amount) { real impl };

    public static void main (String[] args) throws RemoteException,
    AlreadyBoundException {

        Account acc = new AccountImpl(1234, 2000);
        Account accproxy = (Account)UnicastRemoteObject.exportObject(acc);
        Registry registry = LocateRegistry.getRegistry();
        registry.bind("Acc", accproxy);
    }}
```

Implementation Server

- ❑ Declare the remote interfaces being implemented
- ❑ Define the constructor for the remote object
- ❑ Provide an implementation for each remote method in the remote interfaces

Implementation Server

- ❑ The server needs to create and to install the remote objects. This setup procedure can be encapsulated in a main method in the remote object implementation class itself, or it can be included in another class entirely. The setup procedure should
 - ★ Create one or more instances of a remote object
 - ★ make it available to the RMI runtime environment
 - ★ Creates the client proxy code
 - ★ Register at least one of the remote objects with the RMI remote object registry (or another naming service).

Implementation Client

```
import java.rmi.registry.*;

public class client {
    public static void main(String args[]) {

        try {
            Registry registry = LocateRegistry.getRegistry("mimi");
            Account acc = (Account) registry.lookup("Acc");

            float newBal = acc.addtoBalance(20);

        } catch (Exception e) {
            System.err.println("exception: "+e.getMessage());
        }
    }
}
```

Implementation Client

❑ Write the client code.

- ★ It can call the methods specified in the interface.
- ★ Before calling a method of a remote object it must have a remote object reference
- ★ It can retrieve a remote object reference + proxy
 - By calling the RMIregistry on the machine where the remote object resides
 - It received a remote object reference through a previous call

Running the code

❑ Compile the client code: only needs the interface file in order to compile

☆ Does not need the proxy class definition: they are downloaded on the fly!

❑ Start client

☆ Client contacts the binder on the remote machine to find object

☆ The binder returns a remote object reference

☆ The client downloads the proxy class from server dynamically

☆ The client invokes methods in the remote object

Distributed Garbage Collection

- ❑ An object must exist as long as there is a local or remote reference held
- ❑ Whenever the last references is deleted the object should be garbage collected.
- ❑ A classical distributed algorithm problem:
 - ☆ Solution: reference counting
- ❑ 2-level
 - ☆ local proxy garbage collection on each client
 - Whenever a remote object reference enters a process, a proxy is created, when the last reference to this object on the process is deleted, the proxy is deleted
 - ☆ object maintenance on the server
 - whenever there is no proxy anymore **and** there is no local reference, you can delete the object
 - Really?

Distributed Garbage Collection

❑ Algorithm outline:

- ☆ Each server process maintains a list of processes that currently hold a remote object reference
- ☆ Whenever a new proxy is created (a client first requests a reference), the remote reference module on the client first calls a function `addRef` on the server and then creates the proxy
- ☆ When a proxy on a client is no more needed (detected by the local GC of the client), the remote reference module calls `removeRef` on the server
- ☆ When the list is empty and there are no local references the remote object is removed

Distributed Garbage Collection

- ❑ Pair-wise request-reply between remote reference modules
- ❑ Only called when proxies are created/deleted
- ❑ Fault-tolerance has to be addressed
 - ☆ Leases (max. time to live) in case removeRef gets lost